Calidad y Preprocesamiento de Datos

MCIC VÍCTOR MANUEL CORZA VARGAS

MC FERNANDO AVITÚA VARELA

Unidad 4 Limpieza de datos

Enlace de registros (Record linkage)

Enlace de registros

El enlace de registros o vinculación de registros (también conocido como cotejo de datos, resolución de entidad, y muchos otros términos) es la tarea de encontrar registros en un conjunto de datos que se refieren a la misma entidad en diferentes fuentes de datos

Enlace de registros

_id_a	name	address	dob	hair	timestamp
0	Fred Flinstone	301 Cobblestone Wy., Bedrock	-2505/02/02	black	1960/09/30
1	Wilma Flinstone	301 Cobblestone Wy., Bedrock		red	1960/09/30
2	Barney Rubble	303 Cobblestone Wy., Bedrock, 70777	-2507/09/30	blonde	1960/09/30
3	Betty Rubble	303 Cobblestone Wy., Bedrock, 70777	-2509/11/01		1960/09/30
	...				

_id_b	name	address	dob	hair	timestamp
1001	Frederick J. Flinstone	345 Cave Stone Road	-2505/02/02		1959/11/01
1002	Wilma Anna Flinstone	345 Cave Stone Road		red	1959/11/01
1003	Bernard Matthew Rubble	303 Cobblestone Way, Bedrock, 70777		blonde	1960/09/30
1004	Betty McBricker		-2509/11/01	black	1959/11/01
	...				

_id_a	_id_b	name	address	dob	hair	timestamp
0	1001	Frederick J. Flinstone	301 Cobblestone Wy., Bedrock	-2505/02/02	black	1960/09/30
1	1002	Wilma Anna Flinstone	301 Cobblestone Wy., Bedrock		red	1960/09/30
2	1003	Bernard Matthew Rubble	303 Cobblestone Wy., Bedrock, 70777	-2507/09/30	blonde	1960/09/30
3	1004	Betty Rubble	303 Cobblestone Wy., Bedrock, 70777	-2509/11/01	black	1960/09/30
		...				

Enfoques en enlace de registros

Vinculación de datos basada en reglas

- A falta de un identificador común, se puede decidir vincular registros siempre que la concatenación de tres campos sea la misma.
- Otorgar discrecionalmente un peso a cada campo y decidir que dos registros representan la misma entidad si la suma de los pesos de los campos en los que coinciden supera cierto umbral.

Vinculación probabilística

Si, en lugar de dar pesos a los campos de manera determinista, estos pesos se computan con un modelo que tenga en cuenta las capacidades de los identificadores para determinar si hay o no vinculación, estamos ante un tipo de vinculación probabilística.

Las parejas de registros con una probabilidad por encima del umbral se consideran vinculadas y las que quedan por debajo, no lo están.

Modelo Fellegi-Sunter

Fellegi y Sunter definieron un primer algoritmo específico para vincular registros. Funciona asignando ponderaciones específicas (u y m) a cada identificador donde:

- m sería la probabilidad de que el valor del identificador coincida para un par de registros vinculados.
- u refleja la probabilidad de que el valor de un identificador coincida en un par de registros no vinculados

Explicación completa del algoritmo: https://moj-analytical-services.github.io/splink/topic_guides/theory/fellegi_sunter.html

Comparación con técnicas de Machine Learning

- Equivalencia sugerida con clasificador naïve Bayes (Wilson, 2011).
- Algoritmos como regresión logística y máquinas de soporte vectorial pueden ofrecer resultados similares o superiores (Mason, 2018; Aiken et al., 2019).

RecordLinkage

https://recordlinkage.readthedocs.io/en/latest/index.html

Recordlinkage

- Recordlinkage is a powerful Python library primarily designed for data matching and deduplication. It provides a comprehensive set of tools and algorithms to identify and link records that are the same or related across different data sources.
- Recordlinkage utilizes pandas and numpy for data processing and calculations.

Recordlinkage features

- Clean and standardise data with easy to use tools
- Make pairs of records with smart indexing methods such as **blocking** and **sorted neighbourhood** indexing
- Compare records with a large number of comparison and similarity measures for different types of variables such as strings, numbers and dates.
- Several classifications algorithms, both supervised and unsupervised algorithms.
- Common record linkage evaluation tools
- Several built-in datasets.

Importance of attributes in RL

In record linkage, the attributes of the entity (stored in a record) are used to link two or more records.

Attributes can be unique entity identifiers (SSN, license plate number), but also attributes like (sur)name, date of birth and car model/colour.

Record linkage workflow

The record linkage procedure can be represented as a workflow [Christen, 2012]. The steps are:

- Preprocessing (Cleaning)
- Indexing
- Comparing

And in case record linkage is made using Machine learning

- Classifying
- Evaluation

Step 0 Preprocessing

Preprocessing data, like cleaning and standardising, may increase your record linkage accuracy. The Python Record Linkage Toolkit contains several tools for data preprocessing.

Cleaning: The Python Record Linkage Toolkit has some cleaning function. **recordlinkage.preprocessing.clean()** is the most generic function.

```
recordlinkage.preprocessing.clean(s, lowercase=True,
replace_by_none='[^ \\-\\_A-Za-z0-9]+',
replace_by_whitespace='[\\-\\_]', strip_accents=None,
remove_brackets=True, encoding='utf-8')
```

Step 0 Preprocessing

Phonetic encoding

Phonetic algorithms are algorithms for indexing of words by their pronunciation. The most well-known algorithm is the Soundex algorithm. The Python Record Linkage Toolkit supports multiple algorithms through the recordlinkage.preprocessing.phonetic() function.

The implemented algorithms are **soundex, nysiis, metaphone** or **match_rating**.

**Use phonetic algorithms in advance of the indexing and comparing step. This results in most situations in better performance.*

Step 0 Preprocessing

```
from recordlinkage.preprocessing import clean, phonetic

dfA['address_1_cleaned'] = clean(dfA['address_1'],
                    replace_by_none='[^ \\-\\_\(\)A-Za-z0-9]+',
                        remove_brackets=True, strip_accents = 'unicode')

dfA['given_name_methaphone'] = phonetic(dfA['given_name'],
method='metaphone')

dfA['given_name_nysiis'] = phonetic(dfA['given_name'], method='nysiis')
```

Step 0 Preprocessing

given_name	given_name_methaphone	given_name_nysiis
elton	ELTN	ELTAN
mitchell	MXL	MATCAL
NaN	NaN	NaN
elk i	ELK	ELC
NaN	NaN	NaN
...	...	...
connor	KNR	CANAR
daniel	TNL	DANAL
samuel	SML	SANAL
saah	S	S
NaN	NaN	NaN

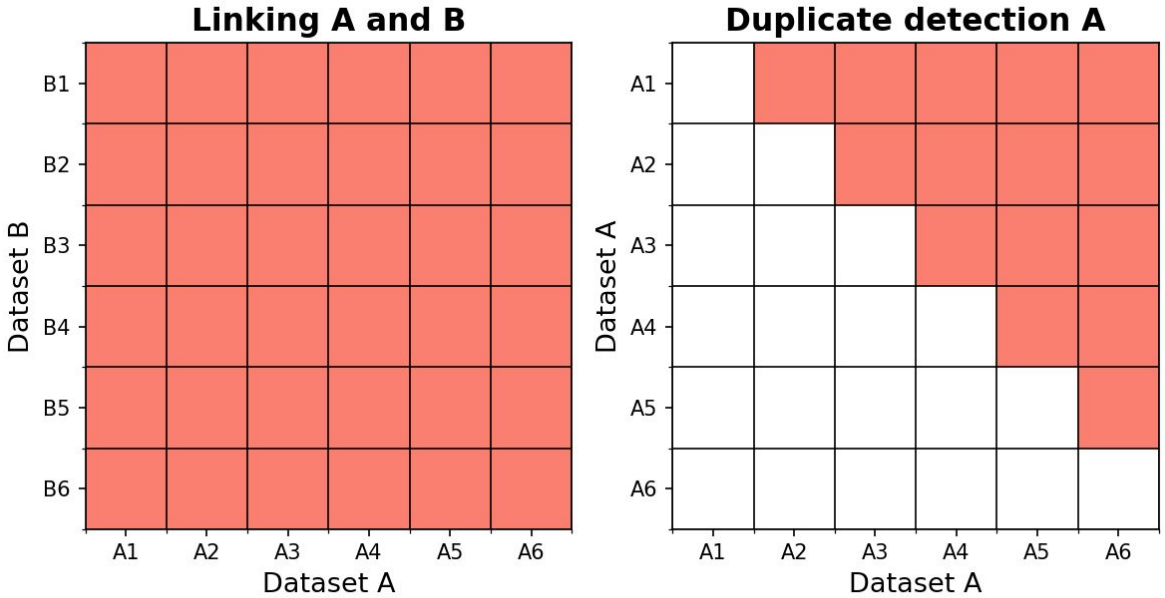
Step 1 Indexing

The indexing module is used to make pairs of records. These pairs are called candidate links or candidate matches. There are several indexing algorithms available such as:

- Full
- Block
- Sorted neighborhood indexing
- Random

Step 1 Indexing

The indexing module can be used for both **linking and duplicate** detection. In case of duplicate detection, only pairs in the upper triangular part of the matrix are returned. This means that the first record in each record pair is the largest identifier. For example, (“A2”, “A1”), (5, 2) and (“acb”, “abc”). The following image shows the record pairs for a complete set of record pairs.



Indexing Algorithms

The Python Record Linkage Toolkit contains basic and advanced indexing (or blocking) algorithms to make record pairs. The algorithms are Python classes. Popular algorithms in the toolkit are:

- recordlinkage.index.Full,
- recordlinkage.index.Block,
- recordlinkage.index.SortedNeighbourhood

Algorithm ‘full’

A full index is an index with **all possible combinations of record pairs**.

In case of linking, this indexation method generates the cartesian product of both DataFrame’s.

In case of deduplicating DataFrame A, this indexation method are the pairs defined by the upper triangular matrix of the A x A.

```
indexer = recordlinkage.Index()
indexer.full()
pairs = indexer.index(census_data_1980, census_data_1990)
```

Algorithm ‘block’

Make candidate record pairs that agree on one or more variables.

Returns all record pairs that agree on the given variable(s). This method is known as blocking. Blocking is an effective way to make a subset of the record space (A x B).

```
indexer = recordlinkage.Index()
indexer.block(left_on='first_name', right_on='givenname')
indexer.index(census_data_1980, census_data_1990)
```

Algorithm ‘sorted neighbourhood’

This algorithm returns record pairs that agree on the sorting key, but also records pairs in their neighbourhood. A large window size results in more record pairs. A window size of 1 returns the blocking index.

The Sorted Neighbourhood Index method is a great method when there is relatively large amount of spelling mistakes. Blocking will fail in that situation because it excludes to many records on minor spelling mistakes.

```
indexer = recordlinkage.SortedNeighbourhoodIndex(
        left_on='first_name', right_on='given_name',
window=9)
indexer.index(census_data_1980, census_data_1990)
```

Step 2 Comparing

A set of informative, discriminating and independent features is important for a good classification of record pairs into matching and distinct pairs. The `recordlinkage.Compare` class and its methods can be used to compare records pairs. Several comparison methods are included such as string similarity measures, numerical measures and distance measures.

recordlinkage.Compare

Class to compare the attributes of candidate record pairs. The Compare class has methods like string, exact and numeric to initialise the comparing of the records. The compute method is used to start the actual comparing.

Ejemplo

Base de datos 1:

ID Cliente	Nombre	Dirección	Teléfono
001	Juan Pérez	Calle 123, Ciudad A	555-1234567
002	María López	Calle 456, Ciudad B	555-2345678
003	Ana García	Calle 789, Ciudad A	555-3456789

Base de datos 2:

ID Cliente	Nombre	Dirección	Teléfono
101	Juan P. Pérez	Calle 123, Ciudad A	555-1234567
102	María Lopez	Calle 456, Ciudad B	555-2345678
103	Ana García Pérez	Calle 789, Ciudad A	555-3456789

recordlinkage.Compare

Consider two historical datasets with census data to link. The datasets are named census_data_1980 and census_data_1990. The MultiIndex candidate_pairs contains the record pairs to compare. The record pairs are compared on the first name, last name, sex, date of birth, address, place, and income:

```
comparer = rl.Compare()

comparer.string('name_a', 'name_b', method='jarowinkler', threshold=0.85, label='name')

comparer.exact('sex', 'gender', label='gender')

comparer.date('dob', 'date_of_birth', label='date')

comparer.string('str_name', 'streetname', method='damerau_levenshtein', threshold=0.7, label='streetname')

comparer.exact('place', 'placename', label='placename')

comparer.numeric('income', 'income', method='gauss', offset=3, scale=3, missing_value=0.5, label='income')

comparer.compute(pairs, dfA, dfB)

# the method .compute() returns the DataFrame with the feature vectors.

comparer.compute(candidate_pairs, census_data_1980, census_data_1990)
```

Algorithms for comparison

classrecordlinkage.compare.Exact

Compare the record pairs exactly. This class is used to compare records in an exact way. The similarity is 1 in case of agreement and 0 otherwise.

classrecordlinkage.compare.String

Compute the (partial) similarity between strings values. This class is used to compare string values. The implemented algorithms are: ‘jaro’, ’jarowinkler’, ‘levenshtein’, ‘damerau_levenshtein’, ‘qgram’ or ‘cosine’. In case of agreement, the similarity is 1 and in case of complete disagreement it is 0.

Algorithms for comparison

classrecordlinkage.compare.Numeric

Compute the (partial) similarity between numeric values. This class is used to compare numeric values. The implemented algorithms are: ‘step’, ‘linear’, ‘exp’ or ‘gauss’. In case of agreement, the similarity is 1 and in case of complete disagreement it is 0.

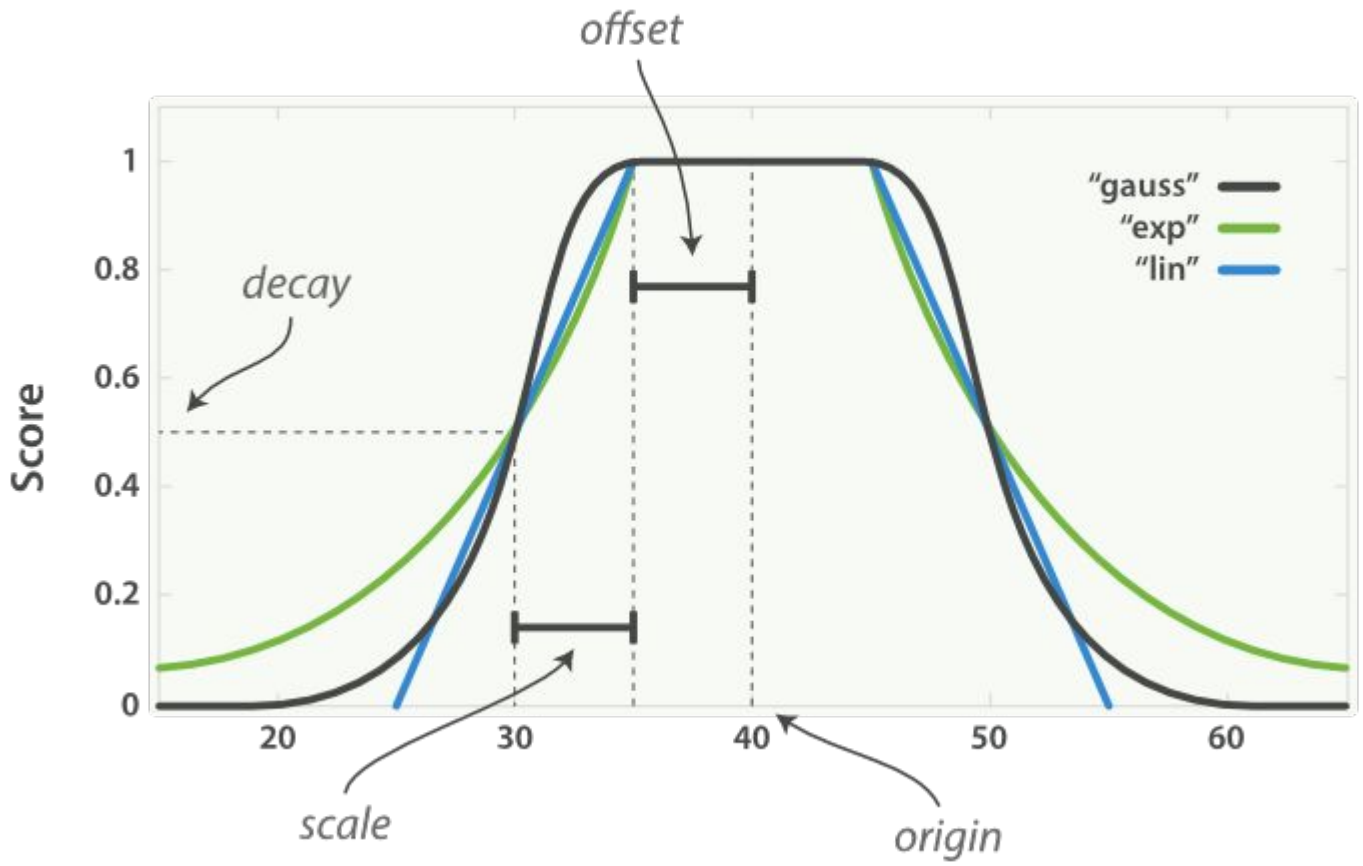
recordlinkage.compare.Geographic()

Compute the (partial) similarity between WGS84 coordinate values. Compare the geometric (haversine) distance between two WGS- coordinates. The similarity algorithms are ‘linear’, ‘exp’ and ‘gauss’. The similarity functions are the same as in recordlinkage.comparing.Compare.numeric()

recordlinkage.compare.Date

Compute the (partial) similarity between date values.

classrecordlinkage.compare.Numeric



https://www.elastic.co/guide/en/elasticsearch/guide/current/decay-functions.html

compute(pairs, x, x_link=None)

Compare the records of each record pair.

Calling this method starts the comparing of records.

```
# the method .compute() returns the DataFrame with the feature vectors.
Features =
comparer.compute(candidate_pairs, census_data_1980, census_data_1990)
```

Last step of compare

The last step is to decide which records belong to the same person.

```
candidate_pairs.sum(axis=1).value_counts().sort_index(ascending=False)
```

7.0 2100

6.0 1678

5.0 1208

4.0 526

3.0 139

2.0 1500

1.0 2700

Name: count, dtype: int64

Step3 Classification

Classification is the step in the record linkage process where record pairs **are classified into matches, non-matches and possible matches** [Christen2012]. Classification algorithms can be **supervised or unsupervised** (with or without training data).

Step3 Classification

There are dozens of classification algorithms for record linkage. Roughly speaking, classification algorithms fall into two groups:

- **Supervised learning algorithms** - These algorithms make use of training data. If you do have training data, then you can use supervised learning algorithms. Most supervised learning algorithms offer good accuracy and reliability. Examples of supervised learning algorithms in the Python Record Linkage Toolkit are Logistic Regression, Naive Bayes and Support Vector Machines.
- **Unsupervised learning algorithms** - These algorithms do not need training data. The Python Record Linkage Toolkit supports K-means clustering and an Expectation/Conditional Maximisation classifier.

Unsupervised methods

K-means clustering

The K-means clusterings algorithm partitions candidate record pairs into matches and non-matches. Each comparison vector belongs to the cluster with the nearest mean.

The K-means algorithm is an unsupervised learning algorithm. The algorithm doesn't need training data for fitting. The algorithm is calibrated for two clusters: a match cluster and a non-match cluster). The centers of these clusters can be given as arguments or set automatically.

The KMeansClassifier classifier uses the sklearn.cluster.KMeans clustering algorithm from SciKit-learn as kernel.

https://developers.google.com/machine-learning/clustering/kmeans/overview?hl=es-419

K-means clustering

```
kmeans = recordlinkage.KMeansClassifier()
result_kmeans = kmeans.fit_predict(features)
len(result_kmeans)
```

All information on K-means clustering at:

https://recordlinkage.readthedocs.io/en/latest/ref-classifiers.html

Supervised methods

First things first (Labeled data)

The examples below make use of the **Krebs register** (German for cancer registry) dataset. The Krebs register dataset contains **comparison vectors of a large set of record pairs**. For each record pair, it is known if the records represent the same person (match) or not (non-match). **This was done with a massive clerical review**.

Krebs register dataset

The dataset contains **5749132 compared record pairs** and has the following variables: first name, last name, sex, birthday, birth month, birth year and zip code. The Krebs register contains **20931** matching record pairs.

```
import pandas
import recordlinkage as rl
from recordlinkage.datasets import load_krebsregister
krebs_X, krebs_true_links = load_krebsregister(missing_values=0)
krebs_X
```

Krebs register dataset

In the example in this section, we consider the first 50,000 record pairs of the Krebs register data as training set.

```
golden_pairs = krebs_X[0:50000]


#(labels)

# 183 matching pairs

golden_matches_index =
golden_pairs.index.intersection(krebs_true_links)


golden_matches_index
```

Supervised Algorithms

In recordlinkage there are three supervised algorithms adapted for record linkage and deduplication.

- Logistic Regression Classifier.
- Naive Bayes Classifier.
- Support Vector Machines Classifier.

Logistic Regression Classifier.

This classifier is an application of the logistic regression model. The classifier partitions candidate record pairs into **matches and non-matches**.

The LogisticRegressionClassifier classifier uses the sklearn.linear_model.LogisticRegression classification algorithm from SciKit-learn as kernel.

Parameters:

- coefficients (list, numpy.array) – The coefficients of the logistic regression.
- intercept (float) – The interception value.

Logistic Regression Classifier.

```
# Initialize the classifier
logreg = recordlinkage.LogisticRegressionClassifier()

# Train the classifier
logreg.fit(golden_pairs, golden_matches_index)
print ("Intercept: ", logreg.intercept)
print ("Coefficients: ", logreg.coefficients)
```

Logistic Regression Classifier.

```
result_logreg = logreg.predict(krebs_X)
len(result_logreg)
```

All information on Logistic Regression at:

https://recordlinkage.readthedocs.io/en/latest/ref-classifiers.html

Step 4 Evaluation

Evaluation of classifications plays an important role in record linkage. Express your classification quality in terms accuracy, recall and F-score based on true positives, false positives, true negatives and false negatives.

Step 4 Evaluation

Confusion matrix.

The confusion matrix is of the following form:

	Predicted Positives	**Predicted Negatives**
True Positives	True Positives (TP)	False Negatives (FN)
True Negatives	False Positives (FP)	True Negatives (TN)

Step 4 Evaluation

F1 Score

The F1 score is the harmonic mean (a kind of average) of precision and recall. This metric balances the importance of precision and recall, and is preferable to accuracy for class-imbalanced datasets.

recordlinkage.fscore(links_true, links_pred)

Compute the F-score.

The F-score is given by **2*(precision*recall)/(precision+recall)**.

Parameters:

links_true (pandas.MultiIndex, pandas.DataFrame, pandas.Series) – The true (or actual) collection of links.

links_pred (pandas.MultiIndex, pandas.DataFrame, pandas.Series) – The predicted collection of links.

Returns: float – The fscore

References

Christen, P. (2012). Data matching: concepts and techniques for record linkage, entity resolution, and duplicate detection. Springer Science & Business Media.

Christen, P. (2008). Febrl - A Freely Available Record Linkage System with a Graphical User Interface.